

Acadêmicos: Maurício Fabiano Azevedo e Eduardo Barboza de Almeida

Dataset: VOC2012 — Classes: cat e horse

Repositório Git: https://github.com/mauricio1804/CBIR_Recuperacao_de_imagem

2. Dataset e Carregamento

2.1 Estrutura do VOC2012

O dataset utilizado para este trabalho foi o Pascal VOC 2012, organizado com imagens JPEG e anotações XML no formato PASCAL VOC. Cada arquivo XML descreve os objetos presentes na imagem correspondente, fornecendo o rótulo da classe e as coordenadas da bounding box (bbox) de cada objeto.

O sistema filtra apenas as classes de interesse.

```
VOC_CLASSES = {'cat', 'horse'}
```

2.2 Leitura das Anotações — parse_annotation

A função `parse_annotation` é responsável por ler o arquivo XML de anotações de uma imagem e extrair os objetos anotados que pertencem às classes de interesse. Para cada objeto encontrado, é extraído o rótulo (label) e as coordenadas da bounding box nos campos `xmin`, `ymin`, `xmax`, `ymax`.

```
def parse_annotation(xml_path):
    root = elementtree.parse(xml_path).getroot()
    objects = []
    for obj in root.findall('object'):
        label = obj.findtext('name')
        if label not in VOC_CLASSES:
            continue
        b = obj.find('bndbox')
        objects.append({
            'label': label,
            'bbox': [int(float(b.findtext(t))) for t in ('xmin', 'ymin',
                                                         'xmax', 'ymax')],
        })
    return objects
```

O retorno é uma lista de dicionários, cada um contendo o rótulo e a bbox do objeto.

2.3 Carregamento do Dataset — load_dataset

A função `load_dataset` percorre os IDs de imagens disponíveis no arquivo `trainval.txt` do VOC2012, carrega suas anotações via `parse_annotation` e seleciona até `MAX_DOCS=100` imagens que contenham pelo menos um objeto das classes de interesse. A lista de IDs é embaralhada com `random.seed(42)` para garantir reprodutibilidade.

Em seguida, o sistema seleciona as `MAX_QUERIES=5` imagens mais conectadas semanticamente, ou seja, aquelas cujas classes aparecem com mais frequência nas demais imagens do conjunto, para servirem de consulta. Isso garante que as queries tenham muitos

documentos relevantes no índice, tornando a avaliação mais informativa.

```
def load_dataset(voc_root, max_docs, max_queries):
    imgs_dir = os.path.join(voc_root, 'JPEGImages')
    ann_dir = os.path.join(voc_root, 'Annotations')
    ids = open(os.path.join(voc_root, 'ImageSets',
                             'Main', 'trainval.txt')).read().split()
    random.shuffle(ids)

    docs = []
    for img_id in ids:
        img_path = os.path.join(imgs_dir, f'{img_id}.jpg')
        ann_path = os.path.join(ann_dir, f'{img_id}.xml')
        if not (os.path.isfile(img_path) and os.path.isfile(ann_path)):
            continue
        objects = parse_annotation(ann_path)
        if not objects:
            continue
        labels = sorted({o['label'] for o in objects})
        docs.append({'image_path': img_path,
                     'objects': objects, 'labels': labels})
        if len(docs) >= max_docs:
            break

    label_map = {d['image_path']: set(d['labels']) for d in docs}
    candidates = sorted(
        [(d, sum(1 for p, l in label_map.items()
                 if p != d['image_path'] and set(d['labels']) & l))
         for d in docs],
        key=lambda x: x[1], reverse=True
    )

    selected_queries = [d for d, n in candidates[:max_queries]]

    print(f'\nQueries selecionadas ({len(selected_queries)}):')
    for q in selected_queries:
        print(f'    • {os.path.basename(q["image_path"])} → {q["labels"]}')

    return docs, selected_queries
```

3. Propostas de Regiões Candidatas

Antes de extrair características visuais, o sistema precisa definir quais regiões de cada imagem serão analisadas. Diferentemente de abordagens baseadas em grade fixa (como a divisão 3×3 empregada em outros sistemas), este trabalho utiliza diretamente as bounding boxes das anotações do VOC2012 como regiões de interesse. Essa escolha é semanticamente mais precisa, pois concentra a análise exatamente nos pixels que contêm os objetos anotados.

3.1 object_proposals

A função `object_proposals` recebe a lista de objetos anotados de uma imagem e retorna apenas as bounding boxes individuais de cada objeto. A condição `i.get('bbox')` garante que objetos sem `bbox` definida sejam silenciosamente ignorados.

```
def object_proposals(objects):
    return [i['bbox'] for i in objects if i.get('bbox')]
```

Cada lista representa as coordenadas `[xmin, ymin, xmax, ymax]` da `bbox` de um objeto individual, preservando a granularidade das regiões anotadas. Essa abordagem contrasta

com estratégias de fusão de bboxes, que perderiam informação ao mesclar objetos distintos numa única região.

4. Extração de Descritores Visuais — SIFT

4.2 sift_descriptors

A função `sift_descriptors` recebe um recorte de imagem (`region`) e o objeto SIFT já inicializado, converte a região para escala de cinza (pois o SIFT opera sobre intensidade de pixels, não cor) e extrai os descritores.

```
def sift_descriptors(region, sift):
    gray = cv2.cvtColor(
        region, cv2.COLOR_BGR2GRAY) if region.ndim == 3 else region
    _, des = sift.detectAndCompute(gray, None)
    return des
```

A variável `des` é uma matriz de forma $(N, 128)$, onde N é o número de keypoints detectados na região. O underscore (`_`) descarta os keypoints em si (suas coordenadas na imagem), pois o sistema utiliza apenas os vetores descritores para a comparação visual. Caso nenhum keypoint seja encontrado, o que pode ocorrer em regiões muito pequenas ou sem textura, a função retorna `None`.

5. Bag of Visual Words (BoVW)

Os descritores SIFT têm 128 dimensões e cada imagem gera uma quantidade diferente deles. Para comparar imagens, é necessário uma representação de tamanho fixo. O BoVW fornece um histograma de tamanho K (o número de palavras visuais) que conta quantas vezes cada padrão visual aparece na imagem.

5.2 build_codebook — construindo o vocabulário visual

A função `build_codebook` recebe todos os descritores coletados de todas as regiões de todas as imagens e agrupa-os em K clusters via K-means. Cada centroide de cluster é uma palavra visual, um padrão médio de textura/forma aprendido a partir dos dados.

```
def build_codebook(descriptors_list, k):
    all_des = np.vstack([d for d in descriptors_list if d is not None])
    kmeans = KMeans(n_clusters=min(k, len(all_des)),
                    random_state=42, n_init=10)
    kmeans.fit(all_des)
    return kmeans
```

O `np.vstack` empilha verticalmente todos os arrays de descritores numa única matriz. Se três regiões geraram 47, 63 e 31 descritores respectivamente, a matriz resultante terá forma $(141, 128)$. O parâmetro `min(k, len(all_des))` evita que o número de clusters supere o número de amostras disponíveis, o que causaria erro no K-means. O parâmetro `n_init=10` instrui o algoritmo a executar 10 inicializações distintas e retornar a melhor, reduzindo a sensibilidade ao estado inicial dos centroides.

Com NUM_WORDS=300, o codebook resultante contém 300 palavras visuais, cada uma representada por um centroide de 128 dimensões. Intuitivamente, cada palavra visual captura um padrão visual recorrente no conjunto de imagens, texturas de pelo, bordas de orelha, gradientes de crinas, entre outros.

5.3 bovw — codificando regiões como histogramas

Uma vez construído o codebook, cada região é representada por um histograma de 300 dimensões. A função bovw recebe os descritores de uma região e, para cada descritor, identifica a palavra visual mais próxima (via kmeans.predict). O histograma contabiliza quantos descritores foram atribuídos a cada uma das 300 palavras visuais e é normalizado pela soma total para eliminar o efeito da quantidade de keypoints.

```
def bovw(descriptors, kmeans):
    if descriptors is None or len(descriptors) == 0:
        return np.zeros(kmeans.n_clusters)
    hist = np.bincount(kmeans.predict(descriptors),
                       minlength=kmeans.n_clusters).astype(float)
    return hist / hist.sum() if hist.sum() > 0 else hist
```

O histograma resultante é a "assinatura visual" da região, um vetor de tamanho fixo que pode ser comparado com qualquer outra região, independentemente do número de keypoints detectados. Regiões com padrões visuais similares terão histogramas similares.

6. Representação Espacial

6.1 bbox_to_spatial — normalização da posição

Para incorporar informação sobre a localização dos objetos nas imagens, o sistema normaliza as coordenadas de cada bbox em relação às dimensões da imagem. A função bbox_to_spatial converte a bbox de pixels absolutos para um vetor de 4 valores normalizados: centro x (cx), centro y (cy), largura (w) e altura (h), todos em [0, 1].

```
def bbox_to_spatial(bbox, shape):
    h, w = shape[:2]
    x1, y1, x2, y2 = bbox
    bw, bh = max(1, x2 - x1), max(1, y2 - y1)
    return np.array([(x1 + bw/2)/w, (y1 + bh/2)/h, bw/w, bh/h], dtype=np.float32)
```

A normalização é essencial porque permite comparar posições de objetos entre imagens de tamanhos diferentes. Um gato ocupando o canto superior esquerdo de uma imagem 800×600 e outro gato na mesma posição relativa numa imagem 300×200 terão vetores espaciais idênticos.

6.2 iou_spatial — Intersection over Union

A função iou_spatial calcula o IoU (Intersection over Union) entre dois vetores espaciais normalizados. O IoU mede a proporção de sobreposição entre duas bboxes: vale 1.0 quando são idênticas e 0.0 quando não se sobrepõem de forma alguma.

```
def iou_spatial(s_a, s_b):
    def to_xyxy(s):
        cx, cy, w, h = s
        return cx - w/2, cy - h/2, cx + w/2, cy + h/2
    ax1, ay1, ax2, ay2 = to_xyxy(s_a)
    bx1, by1, bx2, by2 = to_xyxy(s_b)
    ix1, iy1 = max(ax1, bx1), max(ay1, by1)
    ix2, iy2 = min(ax2, bx2), min(ay2, by2)
    if ix2 <= ix1 or iy2 <= iy1:
        return 0.0
    inter = (ix2 - ix1) * (iy2 - iy1)
    union = (ax2 - ax1)*(ay2 - ay1) + (bx2 - bx1)*(by2 - by1) - inter
    return float(inter / union) if union > 0 else 0.0
```

A função converte internamente os vetores [cx, cy, w, h] para o formato [x1, y1, x2, y2] via to_xyxy para facilitar o cálculo da interseção. A área de interseção é o produto das sobreposições horizontal e vertical. A área da união é a soma das áreas individuais menos a interseção, evitando dupla contagem.

7. Indexação das Imagens

A função index_images percorre todos os documentos do dataset e, para cada imagem, extrai features de cada região proposta. O processo ocorre em duas passagens: na primeira, são coletados todos os descritores SIFT brutos; na segunda (após o treinamento do K-means), cada região recebe seu histograma BoVW.

```
def index_images(docs, sift, num_words, annotations):
    print('Indexando imagens...')
    image_features, all_des = {}, []

    for i, doc in enumerate(docs):
        path = doc['image_path']
        print(f' [{i+1}/{len(docs)}] {os.path.basename(path)}')
        img = cv2.imread(path)
        if img is None:
            print(f'Não foi possível carregar {path}, pulando.')
            continue

        objs = annotations.get(path, [])
        proposals = object_proposals(objs)

        feats = []
        for bbox in proposals:
            x1, y1, x2, y2 = bbox
            region = img[y1:y2, x1:x2]
            if region.size == 0 or (y2 - y1) < 10 or (x2 - x1) < 10:
                continue
            des = sift_descriptors(region, sift)
            if des is not None:
                all_des.append(des)
                feats.append({'bbox': bbox, 'descriptors': des,
                              'spatial': bbox_to_spatial(bbox, img.shape)})
        image_features[path] = feats

    print('Construindo codebook (K-means)...')
    kmeans = build_codebook(all_des, num_words)
    for feats in image_features.values():
        for f in feats:
            f['bovw'] = bovw(f['descriptors'], kmeans)

    return image_features, kmeans
```


O filtro `region.size == 0 or (y2-y1) < 10 or (x2-x1) < 10` descarta regiões degeneradas (fora dos limites da imagem) ou muito pequenas (onde o SIFT não encontra keypoints suficientes). O dicionário `image_features` mapeia o caminho de cada imagem para a lista de suas features, cada uma contendo `bbox`, descritores brutos e vetor espacial normalizado. Após o K-means, cada feature recebe também seu histograma BoVW.

8. Processo de Consulta — `run_query`

A função `run_query` implementa o motor de busca do sistema. Dado o caminho de uma imagem de consulta e seus objetos anotados, ela extrai as features das regiões da query e as compara com todas as imagens indexadas, retornando as `top_k` mais similares.

8.1 Extração das features da query

O processo de extração é idêntico ao da indexação: as `bboxes` dos objetos anotados da query são usadas como propostas, SIFT é aplicado em cada região, e o histograma BoVW é computado usando o codebook já treinado.

```
proposals = object_proposals(query_objects)

q_feats = []
for bbox in proposals:
    x1, y1, x2, y2 = bbox
    region = img[y1:y2, x1:x2]
    if region.size == 0 or (y2 - y1) < 10 or (x2 - x1) < 10:
        continue
    des = sift_descriptors(region, sift)
    if des is not None:
        q_feats.append({'bovw': bovw(des, kmeans),
                        'spatial': bbox_to_spatial(bbox, img.shape)})
```

8.2 Comparação e scoring

Para cada imagem candidata no índice, o sistema testa todos os pares possíveis (região da query × região da candidata) e guarda apenas o melhor score. Isso evita que imagens com muitas regiões obtenham vantagem injusta por acumulação de scores médios.

```
for path, feats in image_features.items():
    if path == query_path or not feats:
        continue

    best_score, best_bbox = -1.0, None
    for qf in q_feats:
        qv = qf['bovw'].reshape(1, -1)
        if np.linalg.norm(qv) == 0:
            continue
        for cf in feats:
            cv_ = cf['bovw'].reshape(1, -1)
            if np.linalg.norm(cv_) == 0:
                continue
            vis = float(cosine_similarity(qv, cv_)[0][0])
            spat = iou_spatial(qf['spatial'], cf['spatial'])
            score = alpha * vis + (1.0 - alpha) * spat
            if score > best_score:
                best_score, best_bbox = score, cf['bbox']

    if best_score >= 0:
        results.append(
            {'path': path, 'score': best_score, 'bbox': best_bbox})
```

9. Avaliação — MAP, Precision e Recall

A função `evaluate` computa três métricas padrão de recuperação de informação para cada query e reporta suas médias sobre todas as queries.

As três métricas são complementares: MAP avalia a qualidade do ranqueamento, Precision avalia a proporção de acertos no topo da lista, e Recall avalia a abrangência da busca. O sistema reporta suas médias sobre as 5 queries ao final da execução.

```
def evaluate(query_docs, image_features, labels, sift, kmeans, top_k=5):
    print()
    aps, precisions, recalls = [], [], []

    for doc in query_docs:
        path = doc['image_path']
        query_label = set(doc['labels'])

        results = run_query(path, image_features, sift, kmeans,
                             top_k=top_k, query_objects=doc['objects'], verbose=False)

        n_rel, prec_sum = 0, 0.0
        for rank, r in enumerate(results, 1):
            if query_label & set(labels.get(r['path'], [])):
                n_rel += 1
                prec_sum += n_rel / rank

        total_rel = sum(1 for p, l in labels.items()
                        if p != path and query_label & set(l))
        denom = min(total_rel, top_k)

        # FIX 5 - Precision@K e Recall@K como métricas complementares ao MAP
        precision_k = n_rel / top_k if top_k > 0 else 0.0
        recall_k = n_rel / total_rel if total_rel > 0 else 0.0
        precisions.append(precision_k)
        recalls.append(recall_k)

        print(f' {os.path.basename(path):<30} labels={sorted(query_label)}')
        print(
            f'      relevantes={total_rel} precision_sum={prec_sum:.3f}', end='')

        if denom > 0:
            ap = prec_sum / denom
            aps.append(ap)
            print(
                f' AP={ap:.3f} P@{top_k}={precision_k:.3f} R@{top_k}={recall_k:.3f}')
        else:
            print(' AP=N/A (sem relevantes no dataset)')

    map_score = float(np.mean(aps)) if aps else 0.0
    mean_prec = float(np.mean(precisions)) if precisions else 0.0
    mean_recall = float(np.mean(recalls)) if recalls else 0.0

    print(f'\n>>> MAP{top_k}      = {map_score:.4f}')
    print(f'>>> Precision{top_k} = {mean_prec:.4f}')
    print(f'>>> Recall{top_k}   = {mean_recall:.4f}')
    return map_score, mean_prec, mean_recall
```

10. Visualização dos Resultados

A função `visualize_results` exibe a imagem de consulta e os top-K resultados lado a lado.

```

def visualize_results(query_path, results, query_bbox=None):
    n = len(results) + 1
    cols = min(3, n)
    rows = int(np.ceil(n / cols))
    fig, axes = plt.subplots(rows, cols, figsize=(5 * cols, 5 * rows))
    axes = np.atleast_1d(axes).ravel()

    def show(ax, path, bbox, title, color='lime'):
        img = cv2.imread(path)
        if img is None:
            ax.axis('off')
            return
        ax.imshow(cv2.cvtColor(img, cv2.COLOR_BGR2RGB))
        ax.set_title(title, fontsize=9)
        ax.axis('off')
        if bbox:
            x1, y1, x2, y2 = bbox
            ax.add_patch(Rectangle((x1, y1), x2 - x1, y2 - y1,
                                   linewidth=2, edgecolor=color, facecolor='none'))

    show(axes[0], query_path, query_bbox,
         f'Query: {os.path.basename(query_path)}', color='deepskyblue')
    for i, r in enumerate(results, 1):
        show(axes[i], r['path'], r.get('bbox'),
             f'#{i} {os.path.basename(r["path"])}\nScore: {r["score"]:.3f}')
    for ax in axes[n:]:
        ax.axis('off')

    fig.tight_layout()
    os.makedirs('resultados', exist_ok=True)
    out = os.path.join('resultados',
                       f'ranking_{os.path.splitext(os.path.basename(query_path))[0]}.png')
    fig.savefig(out, dpi=150, bbox_inches='tight')
    print(f' Figura salva: {out}')
    plt.show()
    plt.close(fig)

```

11. Execução Principal — main

A função main orquestra todo o pipeline, definindo os hiperparâmetros do sistema e executando as etapas em sequência: carregamento do dataset, indexação, consultas e avaliação.

```

MAX_DOCS = 100
MAX_QUERIES = 5
NUM_WORDS = 300
TOP_K = 5
ALPHA = 0.8

```

O parâmetro ALPHA=0.8 reflete uma decisão de design: a similaridade visual (capturada pelo SIFT e BoVW) carrega mais informação discriminativa sobre o conteúdo semântico da imagem do que a posição relativa do objeto. O componente espacial (IoU, peso 0.2) atua como fator de desempate e penaliza matches onde objetos similares aparecem em posições muito distintas.

12. Resultados e Considerações Finais



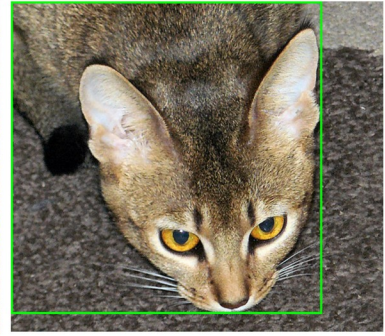
Query: 2009_003663.jpg



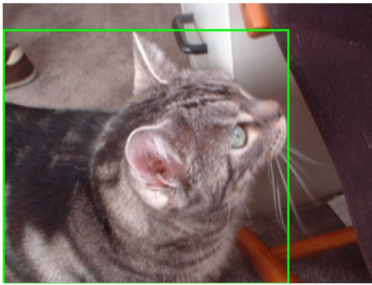
#1 2011_000426.jpg
Score: 0.691



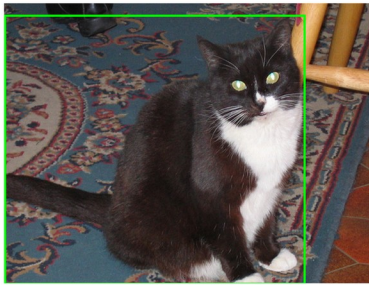
#2 2009_003380.jpg
Score: 0.646



#3 2008_000060.jpg
Score: 0.599



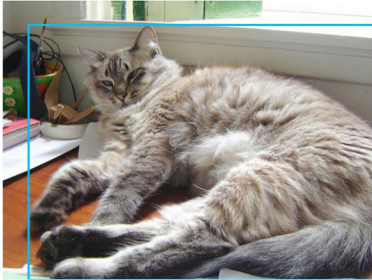
#4 2008_006765.jpg
Score: 0.593



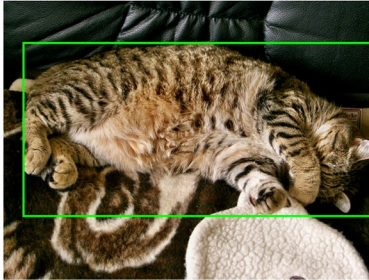
#5 2010_002594.jpg
Score: 0.585



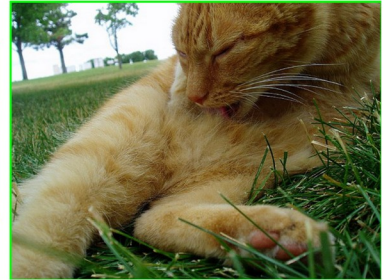
Query: 2010_004963.jpg



#1 2009_005299.jpg
Score: 0.762



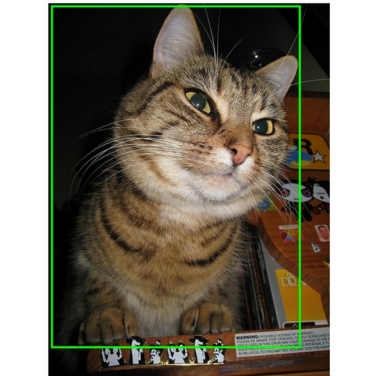
#2 2008_002766.jpg
Score: 0.758



#3 2008_004257.jpg
Score: 0.745

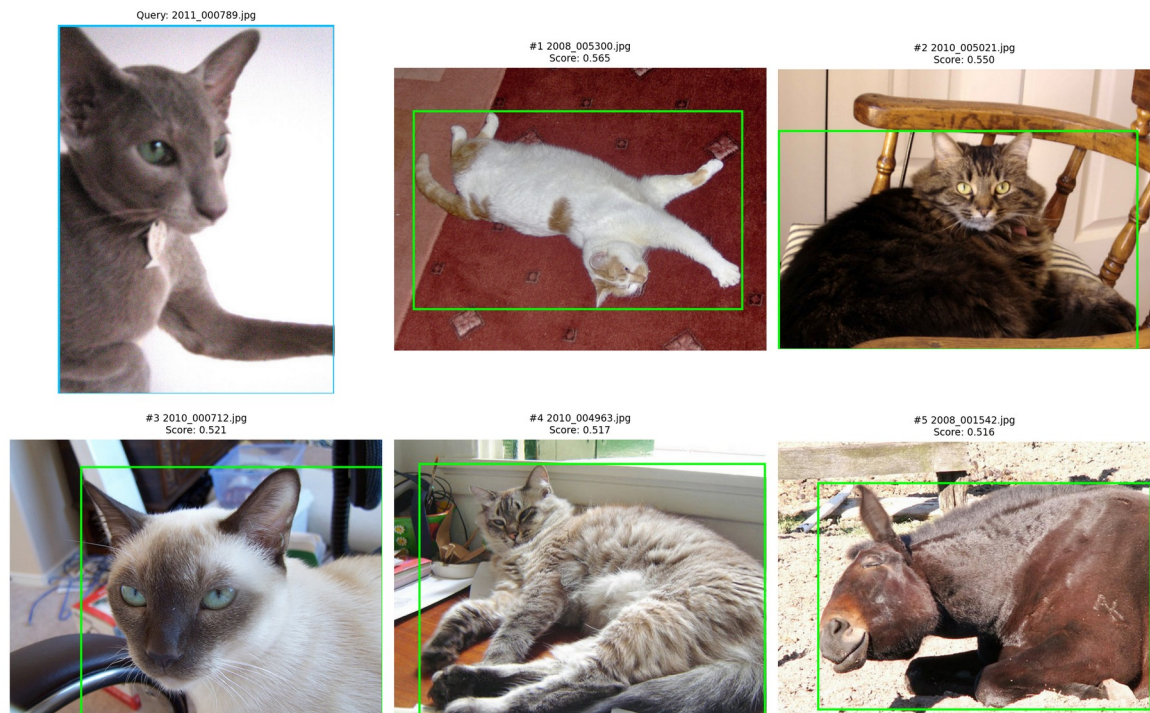


#4 2009_004448.jpg
Score: 0.712



#5 2008_001542.jpg
Score: 0.712





```
=====
AVALIAÇÃO
=====

2011_000789.jpg      labels=['cat']
  relevantes=68  precision_sum=5.000  AP=1.000  PQ5=1.000  RQ5=0.074
2008_005716.jpg      labels=['cat']
  relevantes=68  precision_sum=1.667  AP=0.333  PQ5=0.400  RQ5=0.029
2008_005300.jpg      labels=['cat']
  relevantes=68  precision_sum=5.000  AP=1.000  PQ5=1.000  RQ5=0.074
2009_003663.jpg      labels=['cat']
  relevantes=68  precision_sum=5.000  AP=1.000  PQ5=1.000  RQ5=0.074
2010_004963.jpg      labels=['cat']
  relevantes=68  precision_sum=3.800  AP=0.760  PQ5=0.800  RQ5=0.059

>>> MAP5          = 0.8187
>>> Precision5    = 0.8400
>>> Recall5       = 0.0618
```

O sistema implementado demonstra a viabilidade da abordagem BoVW com SIFT para recuperação de imagens por conteúdo em um subconjunto do VOC2012. Porém demonstrou ser não tão preciso de forma visual, as texturas se aproximam, no entanto acabam sendo confundidas entre animais (gato e cavalo). Observou-se que quando a similaridade visual se sobressai da espacial os resultados são melhores, pois é possível reconhecer a textural e os keypoints com prioridade.